Computación y estructuras discretas I

Unidad 3: Estructuras Discretas no recursivas y análisis de algoritmos – Tablas Hash

Juan Marcos Caicedo Mejía – jmcaicedo@icesi.edu.co

Departamento de Computación y Sistemas Inteligentes
Facultad Barberi de Ingeniería, Diseño y Ciencias Aplicadas
Universidad ICESI
Material adaptado del material original del profesor Oscar Bedoya

2026

Tabla de contenido

1. Lista enlazada

2. Tablas de direccionamiento directo

3. Tablas hash

4. Funciones hash
   - 4.1 Método de división
   - 4.2 Método de multiplicación

5. Implementación en Java

Contenido

1. Lista enlazada

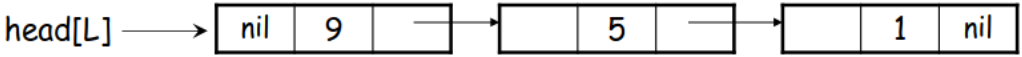
2. Tablas de direccionamiento directo

3. Tablas hash

4. Funciones hash

5. Implementación en Java

Listas simplemente enlazada



Operaciones

- LIST-INSERT(L,x): inserta x al final de la lista. x es un nodo tal que key[x]=k, y prev=next=nil
- LIST-DELETE(L): donde x es el nodo al final de la lista
- LIST-SEARCH(L,k): busca el primer nodo que tiene llave k y retorna un puntero a ese nodo

Contenido

1. Lista enlazada

2. Tablas de direccionamiento directo

3. Tablas hash

4. Funciones hash

5. Implementación en Java

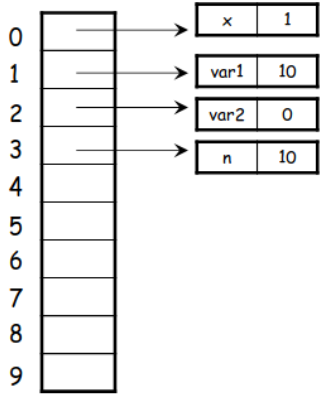
```
programa1(int n)
 x←1
 var1←n
 var2←0
 while (x<n)
    var2←var2 + var1
    x←x+1
 print x
```

Los compiladores llevan una tabla de símbolos cuya llave son los identificadores de las variables

x	1
var1	10
var2	0
n	10

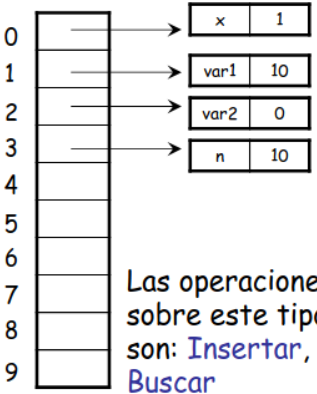
```
programa1(int n)
 x←1
 var1←n
 var2←0
 while (x<n)
    var2←var2 + var1
    x←x+1
 print x
```

Los compiladores llevan una tabla de símbolos cuya llave son los identificadores de las variables

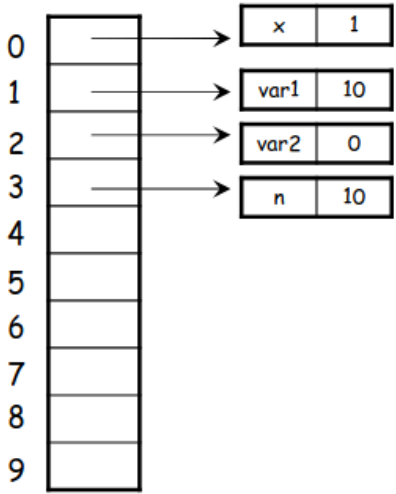


```
programa1(int n)
 x←1
 var1←n
 var2←0
 while (x<n)
   var2←var2 + var1
   x←x+1
 print x
```

Los compiladores llevan una tabla de símbolos cuya llave son los identificadores de las variables

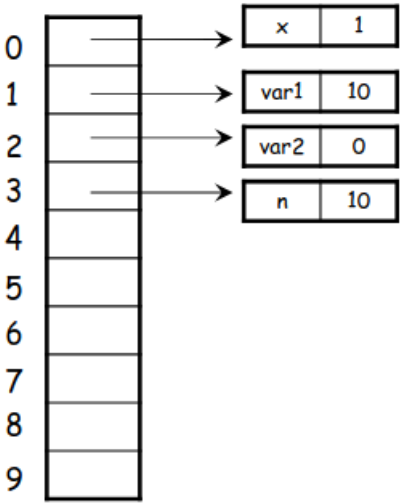


Las operaciones básicas sobre este tipo de tablas son: Insertar, Borrar y Buscar

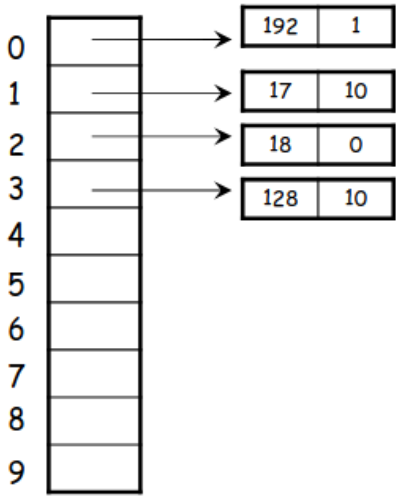


¿Qué tan costoso puede ser insertar un par (llave, valor) en la tabla?

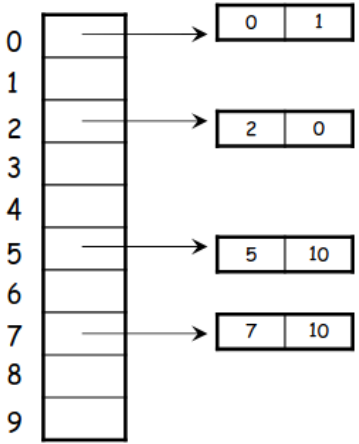
En qué posición de la tabla se debería almacenar un nuevo dato?



¿Qué tan costoso puede ser buscar un par (llave, valor) en la tabla?



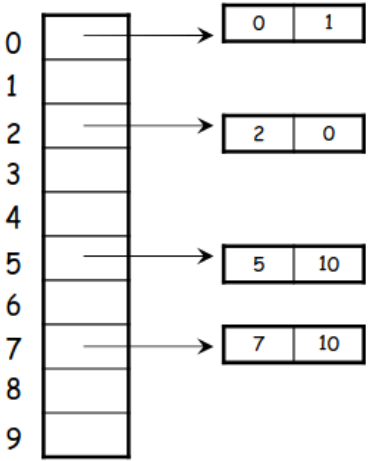
Las llaves se manejan como valores numéricos, en el caso de cadenas de caracteres, se convierten a un número entero utilizando código ASCII



Una estrategia consiste en aprovechar que las llaves son numéricas y almacenar el par (llave, valor) en la posición "llave" de la tabla

Esta estrategia se conoce como Tabla de direccionamiento directo

¿Cuál es el tiempo de búsqueda ahora?



- DIRECT-ADDRESS-INSERT(T,X)

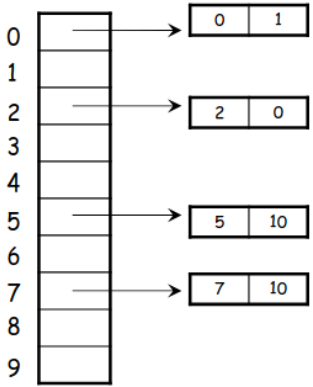
  T[key[x]]←x

- DIRECT-ADDRESS-SEARCH(T,k)

  return T[k]

- DIRECT-ADDRESS-DELETE(T,k)

  T[key[x]]←nil



- DIRECT-ADDRESS-INSERT(T,X)

  T[key[x]]←x

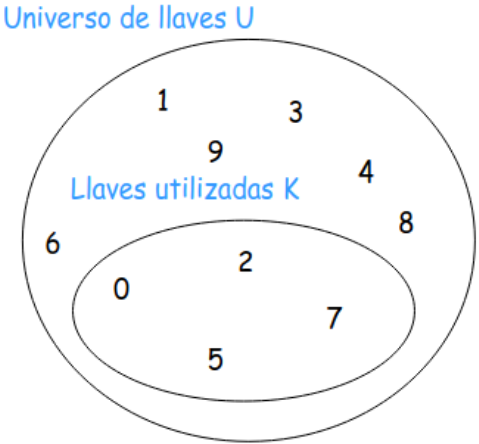
- DIRECT-ADDRESS-SEARCH(T,k)

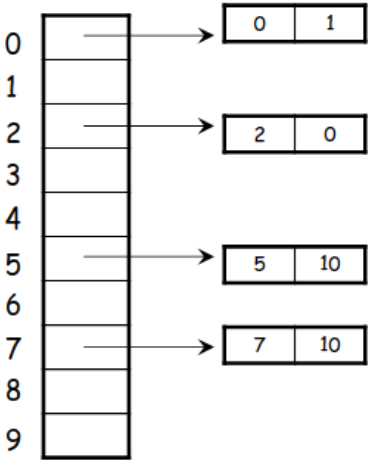
  return T[k]

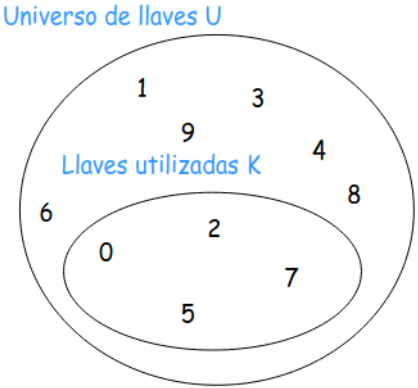
- DIRECT-ADDRESS-DELETE(T,k)

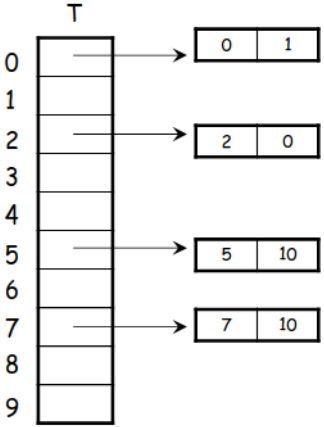
  T[key[x]]←nil

- Todas estas operaciones toman tiempo constante O(1)


Universo de llaves U
1
3
9
4
Llaves utilizadas K
8
6
2
0
7
5


0
1
2
3
4
5
6
7
8
9
0 1
2 0
5 10
7 10





U={0, 1, ..., m-1}, donde |U|=m
La tabla de direccionamiento directo T, se puede ver como un arreglo T[0, ..., m-1] donde cada posición, o slot, corresponde a una llave en U

Tabla de direccionamiento directo T

Considere $K = \{1, 2, 3, 4, 5\}$ el conjunto de llaves actuales, $U = \{0, 1, \ldots, 9\}$ y las siguientes operaciones:

```
DIRECT-ADDRESS-INSERT(T,2)
DIRECT-ADDRESS-INSERT(T,4)
DIRECT-ADDRESS-INSERT(T,3)
DIRECT-ADDRESS-INSERT(T,1)
```

Muestre el contenido de la tabla de direccionamiento directo.

Tablas de direccionamiento directo

Describa un procedimiento para encontrar el elemento máximo de una tabla T de tamaño m. Indique su complejidad.

Tablas de direccionamiento directo

Considere el caso en el que tuviese que almacenar 1000 datos utilizando una tabla de direccionamiento directo.

¿Qué pasa si $|K| \ll |U|$?

Tablas de direccionamiento directo

Considere el caso en el que tuviese que almacenar 1000 datos utilizando una tabla de direccionamiento directo.

¿Qué pasa si $|K| \ll |U|$?

Los requerimientos de memoria pueden llegar a ser de $\mathcal{O}(|U|)$ aun cuando no se utilicen todos los slots.

Tablas de direccionamiento directo

Considere el caso en el que tuviese que almacenar 1000 datos utilizando una tabla de direccionamiento directo.

¿Qué pasa si $|K| \ll |U|$?

Los requerimientos de memoria pueden llegar a ser de $\mathcal{O}(|U|)$ aun cuando no se utilicen todos los slots.

Las tablas hash ofrecen un mecanismo para asignar la posición de almacenamiento para las llaves, de tal forma que los requerimientos de memoria pueden ser de $\mathcal{O}(|K|)$.

Contenido

1. Lista enlazada

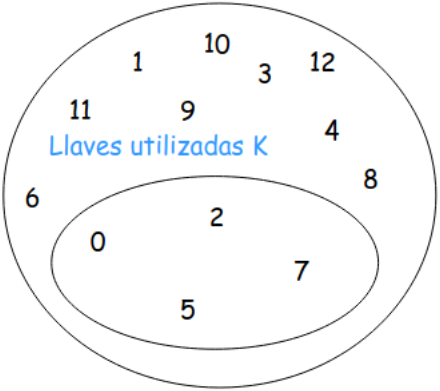
2. Tablas de direccionamiento directo

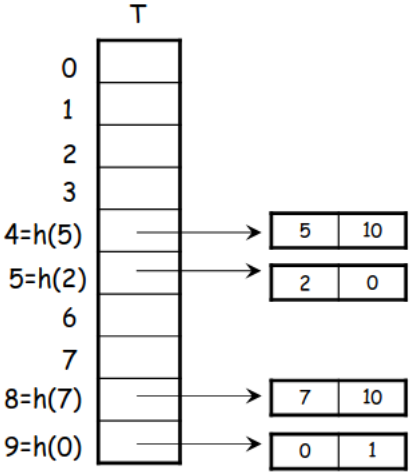
3. Tablas hash

4. Funciones hash

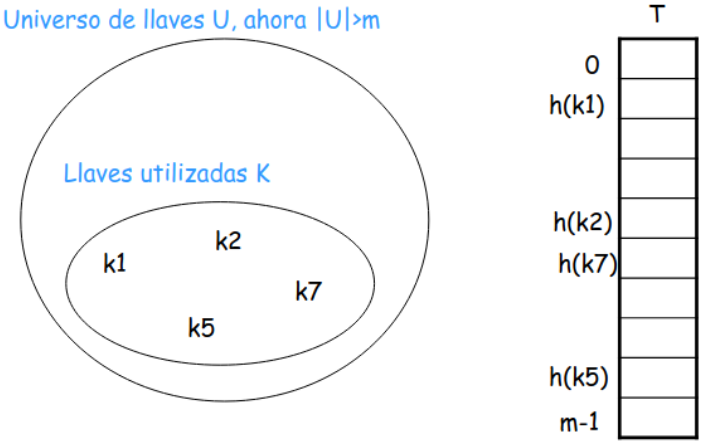
5. Implementación en Java

Universo de llaves U, ahora |U|>m



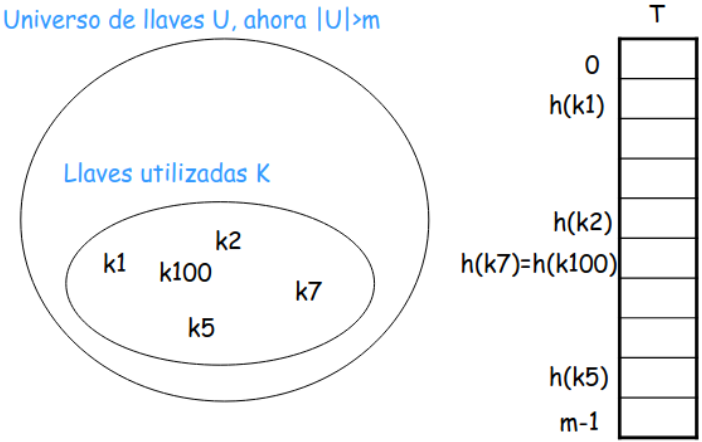


Las tablas hash utilizan una función h: U→{0, 1, ..., m-1}



Las tablas hash utilizan una función h: U→{0, 1, ..., m-1}

La tarea de h es asignar el slot a cada llave



Como |U|>m, pueden existir dos llaves en el mismo slot, esto se llama una **colisión**

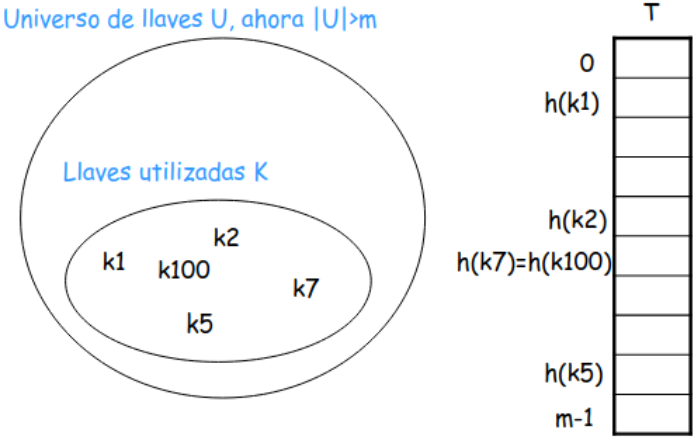
Tablas hash

Tabla hash (suponga que key$(x) = x$ y $m = 5$)

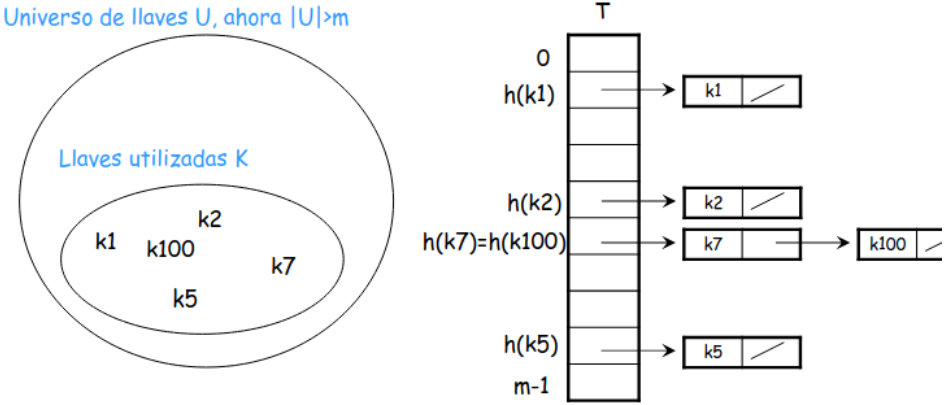
Sea $h(1) = 1, h(4) = 1, h(2) = 3, h(5) = 3, h(3) = 4$

```
HASH-INSERT(T,1)
HASH-INSERT(T,2)
HASH-INSERT(T,3)
HASH-INSERT(T,4)
HASH-INSERT(T,5)
```

Muestre la tabla hash

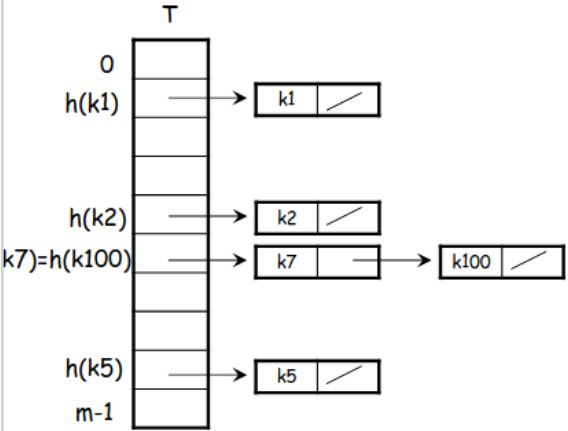


Las colisiones se tratan con diferentes técnicas. La más conocida es la **resolución de colisiones por encadenamiento**



Las colisiones se tratan con diferentes técnicas. La más conocida es la **resolución de colisiones por encadenamiento**

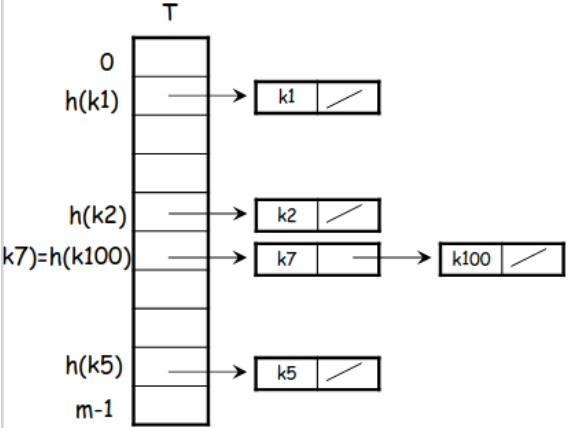
Cada slot T[j] tiene una lista encadenada de todas las llaves cuyo valor hash es j



CHAINED-HASH-INSERT(T,x)
insertar x en la cabeza de la lista T[h(key(x))]

CHAINED-HASH-SEARCH(T,k)
buscar por un elemento con llave k en la lista T[h(key(k))]

CHAINED-HASH-DELETE(T,k)
borrar x de la lista T[h(key(k))]



CHAINED-HASH-INSERT(T,x)
insertar x en la cabeza de la lista T[h(key(x))]

CHAINED-HASH-SEARCH(T,k)
buscar por un elemento con llave k en la lista T[h(key(k))]

CHAINED-HASH-DELETE(T,k)
borrar x de la lista T[h(key(k))]

Analice las complejidades de las operaciones

Tablas hash

Muestra la tabla T después de insertar las llaves $5, 28, 19, 15, 20, 33, 12, 17, 10$ en una tabla hash con 9 slots siendo la función hash $h(k) = k \bmod 9$

Tablas hash

¿Si se mantuvieran ordenados los elementos de cada lista encadenada, cómo cambian los tiempos para insertar, borrar, y buscar?

Contenido

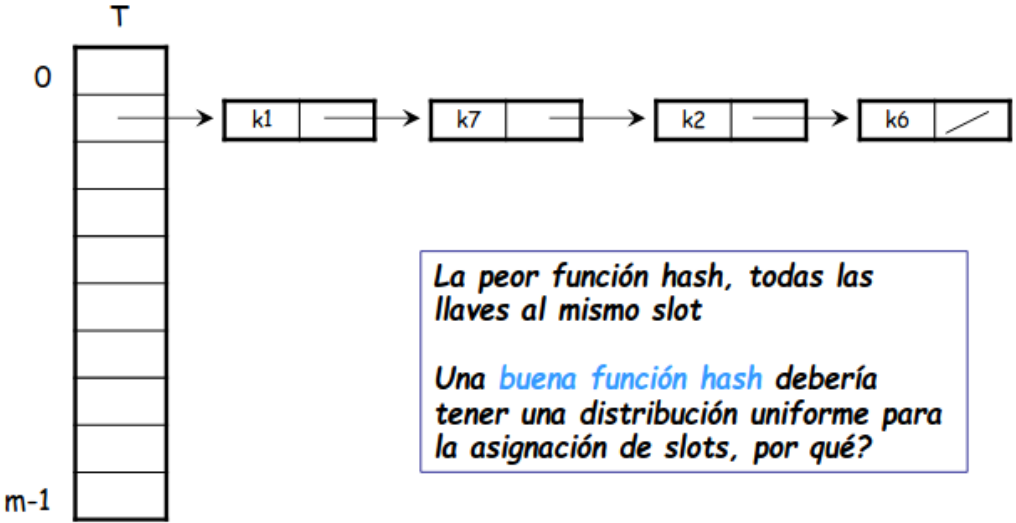
1. Lista enlazada

2. Tablas de direccionamiento directo

3. Tablas hash

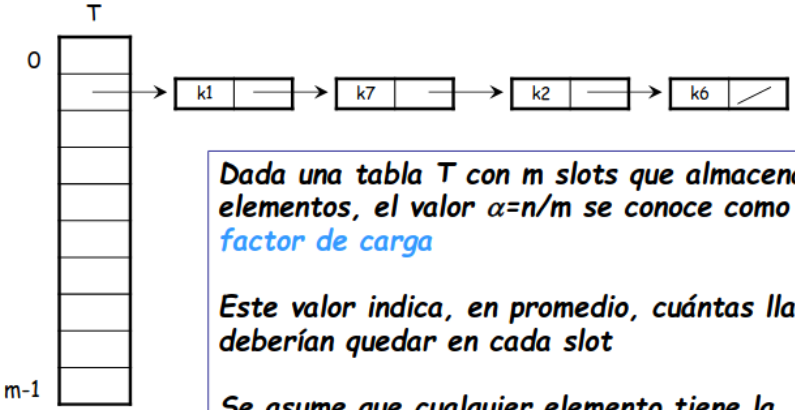
4. Funciones hash
   4.1 Método de división
   4.2 Método de multiplicación

5. Implementación en Java



La peor función hash, todas las llaves al mismo slot

Una buena función hash debería tener una distribución uniforme para la asignación de slots, por qué?



Dada una tabla T con m slots que almacena n elementos, el valor α=n/m se conoce como factor de carga

Este valor indica, en promedio, cuántas llaves deberían quedar en cada slot

Se asume que cualquier elemento tiene la misma probabilidad de ser asignado a cualquiera de los slots, independientemente de donde se hayan asignado otros elementos. Suposición de hashing uniforme

Tablas hash

Teorema 1

En una tabla hash en la cual las colisiones son resueltas con encadenamiento, una búsqueda sin éxito toma en promedio $\Theta(1+\alpha)$, bajo la suposición de hashing uniforme.

Teorema 2

En una tabla hash en la cual las colisiones son resueltas con encadenamiento, una búsqueda exitosa toma en promedio $\Theta(1+\alpha)$, bajo la suposición de hashing uniforme.

Funciones hash

Una buena función hash:

$$\sum_{k:\, h(k)=j} P(k) = \frac{1}{m} \qquad \text{para } j = 0, 1, \ldots, m-1$$

Funciones hash

Es común tener en un programa nombres de identificadores que son similares, `var1`, `var2`, por ejemplo. Una buena función hash debería asignarlos a slots diferentes, así se muestra que existe independencia entre cada par de llaves.

Funciones hash

Llaves de tipo string

Cuando una llave es un string, se utiliza una transformación del código ASCII, en el cual se consideran los caracteres de 0 a 127.

$$\texttt{pt} = 112 \cdot 128^1 + 116 \cdot 128^0 = 14452$$

Funciones hash

Funciones hash

¿Cómo evitar las colisiones o que por lo menos ocurran de tal forma que cualquier colisión sea igual de probable?

Funciones hash

Una función hash

Para el caso en que las llaves sean números reales distribuidos en el rango $0 \leq k < 1$,

$h(k) = \lfloor km \rfloor$, donde $T[0, 1, \ldots, m-1]$

Funciones hash

Una función hash

$h(k) = \lfloor km \rfloor$, donde $T[0, 1, \ldots, m-1]$

Complete la tabla utilizando la fucion:

$$h(k) = \lfloor km \rfloor,$$

para almacenar las llaves

k1=0.4
k2=1.2
k3=1.8
k4=0.9

T

0	
1	
2	
3	
4	

Contenido

1. Lista enlazada

2. Tablas de direccionamiento directo

3. Tablas hash

4. Funciones hash
   4.1 Método de división
   4.2 Método de multiplicación

5. Implementación en Java

Método de división

Método división

Utiliza la función hash:

$$h(k) = k \bmod m$$

Complete la tabla utilizando la función:

$$h(k) = k \bmod m,$$

para almacenar las llaves

k1=4
k2=2
k3=8
k4=9

	T
0	
1	
2	
3	

Complete la tabla utilizando la función:

$$h(k)= k \bmod m,$$

para almacenar las llaves

k1=4
k2=2
k3=8
k4=9

T

0	K1,k3
1	k4
2	K2
3	

A nivel de bits, si m es potencia de 2, se cumple que el valor h(k) dependerá los bits de más bajo orden de k. haciendo que h(k) no dependa de todos los valores de k.

T

0	K1,k3
1	k4
2	K2
3	

Contenido

1. Lista enlazada

2. Tablas de direccionamiento directo

3. Tablas hash

4. Funciones hash

4.1 Método de división

4.2 Método de multiplicación

5. Implementación en Java

Método de multiplicación

Método multiplicación

Utiliza la función hash:

$$h(k) = \lfloor m \cdot (kA \bmod 1) \rfloor \quad \text{donde } A \text{ es cualquier número real entre 0 y 1}$$

El valor de A no es crítico.

Método de multiplicación

Método multiplicación

Sea $m = 10000$, $A = 0.61803$, muestre los valores $h(k)$ que se asignarían para $K = 1000$, 123400, 40321 y 10002.

$$h(k) = \lfloor 10000 \cdot (0.61803 \cdot k \bmod 1) \rfloor$$

Contenido

1. Lista enlazada

2. Tablas de direccionamiento directo

3. Tablas hash

4. Funciones hash

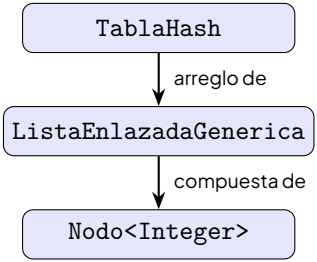
5. Implementación en Java

Implementación en Java

Estructura de la implementación

La tabla hash se construye sobre tres clases:

- `Nodo<T>` — unidad básica con un dato y puntero al siguiente nodo
- `ListaEnlazadaGenerica<T>` — cadena de nodos; resuelve colisiones por *encadenamiento*
- `TablaHash` — arreglo de listas de `Integer`; llave y valor son ambos `int`



Implementación en Java

Nodo.java

```java
public class Nodo<T> {
    public T dato;
    public Nodo<T> siguiente;

    public Nodo(T dato) {
        this.dato = dato;
        this.siguiente = null;
    }
}
```

- T es el tipo genérico: en esta tabla, `T = Integer`
- `siguiente` encadena nodos formando la lista enlazada
- Al crearse, `siguiente = null` indica que es el último nodo

Implementación en Java

ListaEnlazadaGenerica.java

```java
public class ListaEnlazadaGenerica<T> {
    private Nodo<T> cabeza;
    private int size;

    public void agregar(T dato) { ... }
    public T obtener(int indice) { ... }
    public void eliminar(int indice) { ... }
    public int size() { return size; }
    public boolean estaVacia() { return size == 0; }
}
```

- Cada slot de la tabla almacena **una instancia** de esta lista con los valores enteros
- `agregar` recorre hasta el último nodo y enlaza el nuevo: $\Theta(n)$
- `obtener` e `eliminar` también recorren: $\Theta(n)$ en el peor caso
- Con hashing uniforme, cada lista tiene en promedio $\alpha = n/m$ elementos

Implementación en Java

`TablaHash.java` — estructura interna

```java
public class TablaHash {
  private ListaEnlazadaGenerica<Integer>[]
          tabla;
  private int size;

  @SuppressWarnings("unchecked")
  public TablaHash(int n) throws Exception {
    this.tabla = new ListaEnlazadaGenerica[n];
    this.size  = n;
  }

  private int hashFunction(int k)
      throws Exception {
    return k % this.size;
  }
}
```

- `tabla`: arreglo de m listas; cada posición es un *slot* o *bucket*
- Cada lista almacena valores `Integer` directamente — no se necesita clase auxiliar
- `hashFunction`: implementa $h(k) = k \bmod m$

Implementación en Java

TablaHash.java — insert

```java
public void insert(int k, int value)
    throws Exception {
  int pos = hashFunction(k);
  if (tabla[pos] == null)
    tabla[pos] = new ListaEnlazadaGenerica<>();
  tabla[pos].agregar(value);
}
```

- Calcula $h(k) = k \bmod m$
- Crea la lista del slot si aún no existe
- Siempre **agrega** el valor al final — permite duplicados
- Costo promedio: $\Theta(1)$

Implementación en Java

TablaHash.java — search y delete

```java
public boolean search(int k, int value)
    throws Exception {
  int pos = hashFunction(k);
  ListaEnlazadaGenerica<Integer> lista =
      tabla[pos];
  if (lista != null)
    for (int i = 0; i < lista.size(); i++)
      if (lista.obtener(i) == value)
        return true;
  return false;
}

public void delete(int k, int v)
    throws Exception {
  int pos = hashFunction(k);
  ListaEnlazadaGenerica<Integer> lista =
      tabla[pos];
  if (lista == null || lista.estaVacia())
    return;
  for (int i = 0; i < lista.size(); i++)
    if (lista.obtener(i) == v) {
      lista.eliminar(i); return;
    }
}
```

- `search`: recorre el slot buscando el *valor* exacto; retorna `boolean`
- `delete`: localiza el índice `i` y llama `eliminar(i)`
- Ambos: $\Theta(1+\alpha)$ promedio

Implementación en Java

¿Qué es `hashCode()` en Java?

Todo objeto en Java hereda de `Object` el método:

```
public int hashCode();
```

- Para `Integer k`: `k.hashCode()` retorna el propio valor entero k
- Para `String "pt"`: suma ponderada de caracteres ASCII

$$\texttt{"pt"}.\mathit{hashCode}() = 112 \cdot 31^1 + 116 \cdot 31^0 = 3588$$

- Para objetos propios: se debe **sobreescribir** `hashCode()` manualmente
- Contrato: si `a.equals(b)` entonces `a.hashCode() == b.hashCode()`

Implementación en Java

La función hash en esta implementación

```java
private int hashFunction(int k)
    throws Exception {
  return k % this.size;
}
```

- La llave `k` ya es `int`: no se necesita `hashCode()`
- `% this.size` garantiza resultado en $[0,\ m-1]$
- Equivale a $h(k) = k \bmod m$
- Para llaves negativas conviene `Math.abs(k) % size`

Ejemplos con $m = 10$

Llave k	$h(k) = k \bmod 10$
42	2
17	7
130	0

Si se extendiera a llaves genéricas, bastaría reemplazar `k` por `Math.abs(k.hashCode())`.